Viva Guide — Simple Version

Updated 3 November 2026. The same facts as the **Detailed guide** tab, in easy words with examples. Read this one first.

How to use it: read one part, close the screen, and explain it out loud. If you can explain it to a child, you can explain it to the examiner.

1. Our app in one story

Ali, Sara and Bilal go on a trip to Murree.

- Sara pays for the hotel. Ali pays for dinner. Bilal pays nothing.
- At the end everyone asks: “Who owes whom?” Nobody knows.

Our app answers that question. It also helps each person look after their own money at home. So the app has **two rooms**:

Room	It is like	What it does
My money (ExpenseAI)	A money diary	Writes down every rupee you get and spend, warns you when you spend too much, reads shop receipts from a photo, and tells you where things were cheaper
Group money (SmartSplit)	A shared piggy bank for a trip or a flat	Writes down shared bills, works out who owes whom, and settles it with the fewest payments

If the examiner asks “What did you build?”, say: “We built a website for money. One part is a personal money diary that reads receipts with AI and finds savings. The other part splits group bills and settles them in the fewest payments: a group of N people never needs more than N−1 payments.”

Five things that make us special:

1. **Fewest payments.** 5 friends never need more than 4 payments to settle up.
2. **AI reads receipts, but a person checks.** Nothing is saved until you say yes.
3. **Made for Pakistan.** Rupees, JazzCash, Easypaisa, Raast, Pakistani shop prices and zakat.
4. **Money maths is never wrong.** We use whole rupees only, so every split adds up exactly.

5. **Live and tested.** It is on the internet, and 323 automatic checks run before every change goes live.

2. How the parts talk to each other

Think of a restaurant.

- **The website** (what you see in the browser) is the **dining room**. You sit and order here.
- **The server** is the **kitchen**, with a strict **guard at the door**. It does the work and checks who you are.
- **The database** (MongoDB) is the **storeroom**. All the data is kept here.
- **Outside helpers** are the **suppliers**: the AI that reads receipts, Cloudinary that keeps photos, Google for login, a currency-rate service, a gold-price service, and email.

A customer never walks into the storeroom. In the same way, the website never touches the database directly. Every request goes through the server's checks first.

3. Room 1: My money

It is a diary where you write down every rupee.

Example: you get Rs 50,000 salary. You spend Rs 8,000 on food and Rs 15,000 on rent. The app shows: Received Rs 50,000, Spent Rs 23,000, Left Rs 27,000.

What you can do:

- **Add money in or out**, with a title, amount, date, category and payment method.
- **Categories:** Food, Utilities, Travel, Shopping, Healthcare, Housing, Misc, plus your own.
- **Payment methods:** Cash, Bank Transfer, Card, JazzCash, Easypaisa, Raast.
- **Budgets:** "I will spend only Rs 10,000 on Food this month." The bar works like a traffic light: **green** under 80%, **amber** from 80%, **red** at 100%.
- **Recurring bills:** mark rent as monthly. Every night at 2:00 AM the app adds it again by itself. If the server was off for 3 months, it adds all 3 months when it wakes up.
- **Savings goals:** "New laptop, Rs 120,000 by April." Add money over time. The app shows the percent saved, how much to save each month, and whether you are **on track**. On track means: if half the time has passed, you have saved at least half the money.
- **Dashboard:** the home screen. It shows both rooms together.
- **Privacy:** you only see your own data. If someone tries to open your record, the server says "not found", as if it does not exist.

4. Room 2: Group money

A group is a shared piggy bank for a trip, a flat or an event. The person who makes the group is its **admin** (the boss of the group).

Three ways to join a group:

1. **Direct add.** The admin types your email (you already have an account).
2. **Invite link.** For a friend with no account. The admin sends a link on WhatsApp. The friend signs up and is in.
3. **Ask to join.** The group has a share link. You open it and press Ask to join. The admin says yes or no.

Adding a bill: title, amount, date, category, who paid, and how to split it. Two people can pay one bill. Example: hotel Rs 9,000, Sara paid 6,000 and Ali paid 3,000.

Four ways to split a Rs 3,000 dinner for 3 friends:

Way	It is like	Result
Equal	Cutting a pizza into equal slices	1,000 each
Exact	You type each person's amount	Ali 1,500, Sara 1,000, Bilal 500
Percentage	You type percents that make 100%	50% / 25% / 25% = 1,500 / 750 / 750
Line item	Each item goes to the people who had it	Pizza for all three, a drink only for Ali

The extra rupee rule: Rs 100 for 3 people is 33.33 each. We never use decimals. So each person gets Rs 33, and the 1 extra rupee goes to the person who paid: 34 + 33 + 33 = 100. Always exact.

Live checks: if the numbers are wrong, the Save button turns off and the app says why, for example "You're Rs 50 short" or "Percentages add up to 90%, not 100%". The server checks again, so a wrong bill can never be saved.

Trip currency (trips abroad): when you make a group for a Dubai trip, choose **AED** as the trip currency. Every new bill in that group starts in AED, so nobody has to pick it again. The app changes it into rupees with that day's rate.

- Example: a desert safari of AED 412.50 becomes about Rs 31,000. Balances and settle up are always in rupees, and the list still shows "AED 412.5" under the rupee amount.
- There are 15 currencies, such as dirham, riyal, dollar, pound and euro.

Tabs inside a group: Expenses (all the bills), Balances (green = the group owes you, red = you owe the group; all balances add up to zero) and Members.

5. Settle up: the magic trick (most important part)

This is the heart of the project. Every team member must be able to explain it.

Step 1: who is up and who is down? For each person: **net = what they paid − their share.**

Example trip: Sara paid the hotel, Rs 9,000 (Rs 3,000 each). Ali paid dinner, Rs 3,000 (Rs 1,000 each). So each person's share is Rs 4,000.

Person	Paid	Share	Net	Meaning
Sara	9,000	4,000	+5,000	Should get Rs 5,000
Ali	3,000	4,000	−1,000	Should pay Rs 1,000
Bilal	0	4,000	−4,000	Should pay Rs 4,000
Total			0	Always zero

Plus means "should get money". Minus means "should pay money". The nets always add up to zero, because every rupee one person paid, somebody owed.

Step 2: match the biggest with the biggest (greedy).

1. Take the person who should **get the most** and the person who should **pay the most**.
2. The payer pays the smaller of the two amounts.
3. Whoever reaches zero is finished and leaves.
4. Repeat until nobody is left.

Round	Gets most	Pays most	Payment	After
1	Sara +5,000	Bilal −4,000	Bilal → Sara Rs 4,000	Sara +1,000, Bilal done
2	Sara +1,000	Ali −1,000	Ali → Sara Rs 1,000	Everyone done

2 payments instead of 4. The app shows: "4 → 2 payments · 2 transfers saved".

Why is it called greedy? Like a hungry child who always grabs the biggest sweet first. It takes the biggest match right now, without planning ahead.

Why never more than N−1 payments? (the proof, child version) Think of musical chairs. Every payment makes at least one person reach zero, and that person leaves the game. With 5 people, after 4 payments only 1 person is left. All balances add up to zero, so that last person is already at zero. So 5 people need 4 payments at most.

Using it in the app:

1. **Preview:** anyone presses Simplify with Min-Cash-Flow. It shows the payments but saves nothing.
2. **Record:** the group admin confirms, and the payments are saved.
3. **Mark paid:** when someone really sends the money (JazzCash, cash), they press Mark paid.

Tricky questions examiners like:

- **"Is it always the smallest possible number?"** Not always. Finding the true smallest number is a very hard problem, called NP-hard: the computer would have to try every possible mix of people. Greedy is fast and always gives N−1 or fewer. Saying this honestly shows you understand.
- **"How fast is it?"** About O(N^2 log N). For 5 to 20 friends it is instant.
- **"What if the balances don't add up to zero?"** It refuses to run, so a bug can never invent money.
- **"How did you test it?"** The computer made 300 random groups (2 to 10 people) and 100 more with several payers. Every time, all balances reached zero in N−1 payments or fewer.

The code is in server/src/services/settlement.js, about 120 lines.

6. Receipt scanning: the robot reads, you check

OCR means the computer reads words from a picture.

What happens, step by step:

1. You take a photo of a shop receipt (up to 8 MB).
2. The photo is kept on **Cloudinary**, a photo cupboard on the internet.
3. An **AI** reads it: Claude, or a free AI through OpenRouter. It must answer in a fixed form: shop name, date, items (name, quantity, price) and total.
4. If the AI is not available, **Tesseract**, a free reader inside our own server, reads it instead. Like a spare tyre.
5. **Sum check:** the app adds up the items and compares them with the printed total. If they differ by more than 5%, it warns you.
6. **You check every line.** The photo is on the left and the lines are on the right. Fix a wrong price, add a line or delete one.
7. **Save** it to My money, or use it in a group bill.

Why a person must check: AI can read a blurry "1,745" as "1,145". Money must be right, so the rule is: **the AI suggests, the human decides.**

Extra safety: photo links are private and stop working after 1 hour. Each user can scan up to 30 receipts an hour, because the AI costs money.

7. Savings tips and live shop prices

After you save a receipt, the app compares each item with a price list.

Example: you paid Rs 1,890 for Cooking Oil 3L. The list has it for Rs 1,745 at another shop. The app says: “You could save Rs 145.”

How an item on a receipt finds its match:

1. **Clean the name:** “COOKING OIL 3LTR @” and “Cooking Oil 3 Litre” become the same words.
2. **Nicknames:** “anday” means Eggs, “surf excel” means Detergent.
3. **Fuzzy match:** if the words are not exactly the same, the app gives a closeness score from 0 to 1 (the Dice coefficient) and needs 0.8 or more. So “Cookng Oil” still finds “Cooking Oil”.
4. **Same size only:** a 5 kg bag is never compared with a 1 kg bag.
5. **Find cheaper shops:** it shows up to 3 shops that were cheaper.

A real bug we caught (a good story): at first, 5 kg rice was compared with 1 kg rice and showed a fake saving of Rs 1,470. We added the size check to stop it.

Live shop prices (automatic, no Excel): every morning at 9:00 the server visits 5 online shops by itself: **Al-Fatah, GrocerApp, Metro, Jalal Sons and Chase Up**. It writes their latest prices into the price list. Nobody types anything. It also runs when the server starts (if it has not run that day), and from the **Update now** button on the Admin page.

The rules it follows:

- **Only shops that allow it.** Every website has a small rules file called **robots.txt**. It is like a sign on a shop door: “robots welcome” or “no robots here”. Daraz and Naheed say no, so we do not visit them.
- **Polite:** one visit every 2 seconds, and it says its name honestly (SmartSplitFYP).
- **No look-alikes.** Similar names must not get mixed up:
  - “Sugar Cane” is not sugar. “Beef Burger” is not beef. “Tea Cake Rusk” is not tea.
  - Cat food with beef is not beef. A “1 kg × 2” pack is not one 1 kg pack.
  - “Face Wash Tea Tree” is a face wash, not tea, because Face Wash is another product on our list.
  - But “Burger Cheese Slices” is still cheese, because “burger” comes before “cheese”.

- **A sensible price:** it must be between 0.6 and 1.6 times what we already know. We take the **middle** price, not the cheapest, because the cheapest was often a lower grade, like broken rice.
- **If a shop is down:** after 3 failures in a row, the app tries that shop once more 5 minutes later. If it is still down, the old prices stay. Nothing breaks for the user.
- **If a shop stops selling something:** the old price from that shop is removed, so we never show a price that is no longer real.

8. Zakat calculator

The Zakat calculator is on the Goals page.

The simple rule: take what you own and subtract what you owe. If that is at least the **nisab**, zakat is **2.5%**, which is one part in 40.

- **Nisab** = the value of 52.5 tola of silver (the usual choice in Pakistan), or 7.5 tola of gold.
- The app fills in what it already knows: your savings goals and your group balances. You add cash, bank money, gold, silver, business stock and debts.
- Gold and silver prices come by themselves from the free Gold API. An app admin can type in the bazaar (sarafa) price.
- The answer is rounded **up**, so it is never short. Nothing is saved until you press **Record as paid**.

Example: you own Rs 732,000 and owe Rs 3,520. 732,000 − 3,520 = Rs 728,480. That is more than the silver nisab (Rs 157,500 at Rs 3,000 a tola). Zakat = 728,480 ÷ 40 = **Rs 18,212**.

9. Reports, files and the bell

- **Reports page:** a chart of money in and out over 6 months, a donut chart of where the money went, and budget against real spending.
- **Group reports:** total spent, who paid against who used (the fairness chart), and how many payments settle up saved.
- **Downloads:** CSV (a simple spreadsheet), Excel, and PDF for printing.
- **The bell (notifications):** tells members when a bill is added, someone joins, a settlement is made, or a payment is marked paid. If a notification fails, the money action still works.

10. Safety, explained simply

Passwords: the blender. We never keep your real password. We put it in a blender called **bcrypt** and keep only the smoothie, called the **hash**. You cannot turn a smoothie back into fruit. When you log in, we blend what you typed and compare the two smoothies. Blending is slow on purpose (cost 12), so a thief cannot try millions of guesses.

Staying logged in: the cinema. You get two things:

Thing	It is like	Lasts	Kept where
Access token	A ticket for one show	15 minutes	Only in the page's memory
Refresh token	A membership card that gets you new tickets	7 days	A special cookie that scripts cannot read (HttpOnly)

- Every time the membership card is used, you get a **new card** and the old one stops working. This is called **rotation**.
- If someone uses an **old card** again, it was probably stolen, so we log that user out everywhere.

Google login: Google tells us "yes, this is really Ali". We never see the Google password.

Forgot password: we email a link that works once and stops after 30 minutes. We give the same reply even when the email is not registered, so strangers cannot find out who has an account.

The guard at the door: on every group request the server checks: "Are you a member of this group?" It never trusts what the browser says, because a user can change their browser.

More locks:

- **Zod** checks that every piece of incoming data has the right shape, like a form that only accepts numbers in the phone box.
- **Rate limits:** you can try to log in only 60 times in 15 minutes. This stops password guessing.
- **Helmet and CORS:** extra safety settings, and only our own website may call our server.
- **Secrets** (keys) are kept outside the code. The server will not start if one is missing.

11. The tools we used (MERN), in simple words

MERN = **M**ongoDB + **E**xpress + **R**eact + **N**ode.js. All four use one language: JavaScript.

Tool	It is like
React	LEGO blocks for building screens. Each block (a button, a form) is a **component**
Vite	A fast oven that bakes the website
Tailwind CSS	Ready-made paint and clothes for the screens
Recharts	The crayons that draw the charts
Axios	The waiter who carries messages between the website and the server
Node.js	Lets JavaScript work in the kitchen (the server)
Express	The kitchen's order book: which address does what (the API)
MongoDB Atlas	The storeroom on the internet (the database)
Mongoose	Rules for how each box in the storeroom must look
JWT, bcrypt, Helmet	The tickets, the password blender and the safety settings
Zod	The checker at the gate for incoming data
Claude / OpenRouter AI	The robot that reads receipts
Tesseract.js	The spare robot inside our own server
Cloudinary	The photo cupboard
node-cron	An alarm clock that runs jobs on time (bills at 2:00 AM, prices at 9:00 AM)
Vitest, Supertest	The teachers who mark our homework (the tests)
Render	The building where our app lives on the internet
GitHub Actions	The gatekeeper that runs all tests before any change goes live
Brevo / Gmail API	The post office for emails

Why MongoDB and not MySQL? One bill holds lists inside it: several payers, several shares, several items. MongoDB keeps all of that together in one document, like one envelope with all the papers inside. One read gets everything.

Why REST and not GraphQL? REST is like a menu with fixed dishes: one address for groups, one for expenses. It is simpler to build, test and explain.

12. What the database keeps

The database has 16 drawers, called collections. The important ones:

- **Users:** name, email, the password smoothie (hash), Google ID.
- **Groups:** name, type, currency, members (admin or member).
- **Group expenses:** the bill, who paid, each share and the items. The database refuses a bill whose shares don't add up to the total.
- **Settlements:** one planned payment: who pays whom, and how much.
- **Personal expenses, budgets, categories, goals:** your money diary.
- **Receipts:** the photo link, the lines read, and who read it (AI or Tesseract).
- **Benchmark prices:** the price list from the shops.
- **Price sync runs:** a diary of each morning's shop visit, shown on the Admin page.
- **Notifications and audit log:** the bell messages, and a permanent diary of important group actions.

Soft delete: when you delete a bill it is hidden, not thrown away, like moving it to a "deleted" folder.

13. Our golden rules

Rule	Simple meaning	Why
Whole rupees only	Rs 4,200, never 4,200.50	Computers are bad at decimals: 0.1 + 0.2 = 0.30000000000000004. Whole numbers never go wrong
Splits always add up	The shares equal the total, exactly	Checked in the browser, on the server and in the database
Extra rupee to the payer	100 ÷ 3 = 34 / 33 / 33	One fair rule everywhere
Balances are worked out, not saved	Calculated fresh from the bills every time	A saved total can go wrong if one update fails
The server checks membership	The guard checks every time	A user can change their browser

Rule	Simple meaning	Why
AI never saves by itself	A human checks first	Wrong money data is worse than no data

14. Testing, and where the app lives

323 automatic tests in 24 files, all passing on 3 November 2026. A test is a small check the computer runs by itself, like a teacher marking homework.

- **Unit tests (Vitest)** check one small piece. Example: “Rs 100 for 3 people gives 34 / 33 / 33.”
- **API tests (Supertest)** pretend to be the browser and ask the server something. Example: someone outside the group asks for its bills, and the server says no (403).
- **Property tests:** the computer invents 300 random trips and checks the rules always hold.
- The tests use a pretend database, so real data is never touched.

Bugs we found and fixed (good stories):

- 5 kg rice was compared with 1 kg rice and showed a fake saving.
- Changing only a bill’s title wiped its split.
- On a phone, opening a group link while logged in crashed the page.
- Live prices once picked “Sugar Cane” for sugar and cat food for beef. Now look-alikes are blocked.

Where it lives:

- **Website and server:** Render, at https://smarsplit.onrender.com. On the free plan it goes to sleep when nobody uses it, so open it 5 minutes before the viva to wake it up.
- **Database:** MongoDB Atlas (free).
- **Automatic updates:** every change goes to GitHub, GitHub Actions runs all 323 tests, and Render puts it live only if they all pass.
- **Email:** emails go out through Gmail (the Gmail API). We tested it on the live site on 3 November: the invite email arrived. If an email ever fails, invites can still be shared by link or WhatsApp.

15. What it cannot do yet (be honest)

- Settle up gives N−1 or fewer payments, but not always the very smallest number.

- Only 5 shops are live. Daraz and Naheed say no robots, and Imtiaz needs you to pick a branch first.
- Mark paid is done by hand. The app does not send money itself.
- Blurry receipts need more fixing by hand.
- A short name with one typo may not match.
- The free server sleeps, so morning jobs can run late.
- It is a website only, with no phone app, but it works on phone screens.
- Foreign-currency bills use the day's market rate, not your bank's exact rate.

Future ideas: more shops, JazzCash and Easypaisa payment buttons, a phone app, automatic category guesses, and official government prices (PBS) every week.

16. Questions and simple answers

Say each answer in your own words. Short is fine.

About the project

Q1. What problem does it solve? People forget where their money goes, and friends argue about who owes whom after a trip. Our app keeps a money diary with AI help, and settles group money in the fewest payments.

Q2. Who will use it? Students, young workers, flatmates and families in Pakistan.

Q3. How is it different from Splitwise? Personal money and group money in one app, AI receipt reading, Pakistani shop prices, rupees, JazzCash, Easypaisa, Raast and zakat.

Q4. What was the hardest part? Keeping money exactly right everywhere. We used whole rupees, one rounding rule and lots of tests.

The settle-up algorithm

Q5. Explain it. Find each person's net (paid minus share). Match the person who should get the most with the person who should pay the most. The payer pays the smaller amount. Whoever reaches zero leaves. Repeat.

Q6. Why greedy? It always takes the biggest choice right now, like a child grabbing the biggest sweet first, without planning ahead.

Q7. Prove N−1. Each payment makes at least one person zero, and they leave. After N−1 payments one person is left, and because everything adds up to zero, they are already at zero.

Q8. Is it always the minimum? No. The exact minimum is NP-hard, far too slow to find for big groups. Greedy is fast and always gives N−1 or fewer.

Q9. How fast is it? About O(N² log N). For a trip group it is instant.

Q10. Rs 100 for 3 people? 33 each, and the 1 extra rupee to the payer: 34 + 33 + 33 = 100.

Database and server

Q11. Why MongoDB? A bill holds lists (payers, shares, items). MongoDB keeps them together in one document, like one envelope.

Q12. Why not store balances? A saved total can go wrong if one update fails. We work it out fresh from the bills every time.

Q13. What is a REST API? A list of addresses the website calls. GET = read, POST = add, PATCH = change, DELETE = remove. Ours start with /api/v1.

Q14. What is middleware? A check that runs before the main work, like the guard who checks your ticket before you enter the cinema.

Q15. How do you check input? With Zod. Each address has a description of what is allowed, and anything else is rejected with a clear message.

Safety

Q16. How are passwords kept? As a bcrypt hash: a one-way smoothie of the password, slow on purpose (cost 12).

Q17. What is JWT, and why two tokens? JWT is a signed digital pass. The short one (15 minutes) is like a cinema ticket. The long one (7 days) is like a membership card, kept in a cookie that scripts cannot read.

Q18. What is rotation? Each time the membership card is used, you get a new one. If an old card is used again, it was stolen, so we log the user out everywhere.

Q19. How do you stop someone seeing another group? On every group request the server checks that you are an active member. It never trusts IDs from the browser.

Q20. How do you stop password guessing? Rate limits: 60 login tries per 15 minutes, and 30 scans per hour.

Q21. What is XSS? A bad person hides a script inside a page. React makes text safe by default, Helmet adds safety settings, and our tokens are never in localStorage, where such a script could read them.

AI and receipts

Q22. What is OCR? The computer reading words from a picture.

Q23. Which AI? Claude (vision) when its key is set, otherwise a free vision AI through OpenRouter, with Tesseract as the last backup. The AI must answer in a fixed form: shop, date, items, total.

Q24. What if the AI is down? Tesseract, inside our own server, takes over by itself.

Q25. What if the AI reads a wrong price? The sum check warns when the items and the total differ by more than 5%, and the user checks every line before saving.

Q26. How does price matching work? Clean the name, check nicknames, fuzzy match (score 0.8 or more), same size only, then show cheaper shops.

The website

Q27. What is React, and what is a component? React builds screens from LEGO blocks. A component is one block: a button, a form, a table.

Q28. What are hooks? Small helpers such as useState (to remember something) and useEffect (to do something when it changes).

Q29. How does the website talk to the server? Through Axios, our waiter. All the calls live in one folder. It adds the login ticket by itself and gets a new one when it runs out.

Testing

Q30. How did you test it? 323 automatic tests: unit tests (Vitest), API tests (Supertest) and property tests with hundreds of random groups.

Q31. What is property testing? The computer invents many random examples and checks that a rule always holds, such as “everyone reaches zero in N−1 payments or fewer”.

Q32. Tell us about a bug you found. 5 kg rice was compared with 1 kg rice and showed a fake Rs 1,470 saving, so we added a size check. Another: live prices picked Sugar Cane for sugar; now look-alikes are blocked.

Q33. What would you add next? Official PBS prices, JazzCash and Easypaisa payment buttons, a phone app and automatic categories.

New features and going live

Q34. How do foreign-currency bills work? Pick a currency like AED. The app gets that day's rate, turns the bill into whole rupees, and splits it with the same extra-rupee rule. The AED amount is kept for display.

Q35. Why keep balances in rupees? One currency keeps settle up simple and the N−1 promise true. In Pakistan people pay each other back in rupees anyway.

Q36. What if the currency service is down? The last good rates are saved in the database and used. Only if no rate was ever saved does the app say “rates unavailable”.

Q37. How do you know a goal is on track? If half the time has passed, you should have saved at least half the money. Monthly amount = what is left ÷ months left, rounded up.

Q38. How is it put online? One Render service runs the website and the server at one address, with MongoDB Atlas as the database. Every change runs the 323 tests first and goes live only if they pass.

Q39. Why not Vercel for the website? The login cookie only goes to the same website. Vercel and Render would be two different websites, so users would be logged out on every refresh. One address fixes that.

Q40. Super admin or group admin? A group admin runs one group. A super admin (set on the server in ADMIN_EMAILS) runs the whole app: the price list and the Admin page. Nobody can make themselves a super admin from the browser.

Q41. Do emails work? The code sends 4 emails (password reset, two kinds of invite, and join request), and all were tested. On the live site they go through Gmail over HTTPS, because Render′s free plan blocks the normal email ports and Brevo never switched on our account. We tested it live: the invite email arrived. A failed email never breaks the action.

Q42. How does zakat work? What you own minus what you owe now. If that is at least the nisab (52.5 tola of silver or 7.5 tola of gold), zakat is 2.5%: divide by 40 and round up. It is an estimate, so the page says to confirm with a scholar.

Q43. Where do gold and silver prices come from? The free Gold API gives the world price in dollars per ounce. We change it into rupees per tola (1 tola = 11.664 grams) with the day′s dollar rate. An admin can type in the bazaar price.

Q44. How do shop prices update by themselves? Every morning at 9:00 a job visits Al-Fatah, GrocerApp, Metro, Jalal Sons and Chase Up. It finds each product with the same name and size, takes the middle price, and saves it to the price list.

Q45. Why not Daraz? Isn′t this scraping? We only visit shops whose robots.txt (the “robots welcome?” sign) says yes, and we check it every time. Daraz and Naheed say no, so we don′t go there. One visit every 2 seconds, an honest name, only public prices, no login. If the sign cannot be read at all, we skip that shop for the day; that is the official robots.txt rule (RFC 9309).

Q46. How do you stop wrong matches? Same size (within 2%), every word of the name, a sensible price (0.6 to 1.6 times), and the middle price. Look-alikes are blocked: Sugar Cane is not sugar, Beef Burger is not beef, cat food is not beef, and Face Wash Tea Tree is not tea. When a shop stops selling an item, its old price is removed.

Q47. What if a shop's website is down? After 3 failures in a row the app tries once more 5 minutes later. If it is still down, the old prices stay. The other shops carry on, and the user never sees an error.

17. Demo steps (about 8 minutes)

Before the viva:

- Open https://smarsplit.onrender.com 5 minutes early to wake it up.
- Keep a clear printed receipt ready, plus a photo of it on the laptop.
- On the live site, use your own account. Never run the seed command there, because it deletes everything.
- Open Admin → Live store prices. Today's run should say done. If not, press **Update now** 15 minutes before.
- Keep server/src/services/settlement.js open in case they ask to see the code.

Show it in this order:

1. **Login** (30 s): email login; mention Google login and the password blender.
2. **Dashboard** (1 min): money owed, this month's spending, savings found.
3. **A trip group** (1.5 min): the bills, then Balances: green and red, all adding up to zero.
4. **Add a bill** (1 min): type percents that make 90%. Show the message and the Save button turned off. Fix it and save.
5. **Settle up** (2 min): the big moment. Press Simplify with Min-Cash-Flow, show the payments and the transfers saved, record it, and mark one paid.
6. **Scan a receipt** (1.5 min): upload, fix one line, save. Show the savings card.
7. **My money** (30 s): the budget traffic lights and a recurring bill.
8. **Reports** (30 s): the charts, then download a PDF.

If there is time: Goals and Zakat, then a trip in AED.

If something breaks live: stay calm. Say what should happen, then show the tests (npm test in the server folder, 323 passing). Examiners like a team that can explain a problem.

18. Hard words made easy

Word	Simple meaning
Algorithm	A recipe: the steps the computer follows

Word	Simple meaning
API	The list of addresses the website uses to talk to the server
Frontend / Backend	The dining room / the kitchen
Database	The storeroom where data is kept
Hash	A one-way smoothie of data; you can't get the fruit back
Token (JWT)	A signed digital pass that proves who you are
Cookie (HttpOnly)	A small note the browser keeps; HttpOnly means scripts can't read it
Middleware	A check before the main work, like a guard at the door
Greedy	Always take the biggest choice right now
NP-hard	So hard that no fast exact way is known
OCR	Reading words from a picture
Vision model	An AI that understands pictures
Human-in-the-loop	A person checks the AI before it counts
Fuzzy match	Matching words that are nearly the same
Integer	A whole number, no decimals
robots.txt	A website's sign saying what robots may visit
Rate limit	A cap on how many tries you get in a set time
CI/CD	Every change is tested by itself, then put live only if it passes
Soft delete	Hide it instead of throwing it away
Nisab	The least wealth before zakat is due
Tola	A weight for gold and silver; 1 tola = 11.664 grams
Seed	Filling the app with demo data